```
  import * as THREE from './vendor/three/three.module.js';
import { OrbitControls } from './vendor/three/OrbitControls.js';
import { GLTFLoader } from './vendor/three/GLTFLoader.js';
import { DRACOLoader } from './vendor/three/DRACOLoader.js';

const canvas = document.querySelector('#threeViewer');
const progress = document.querySelector('.three-progress span, .model-progress span');
const progressBox = document.querySelector('.three-progress, .model-progress');
const progressBoxes = document.querySelectorAll('.three-progress, .model-progress');
const stats = document.querySelector('.model-stats');
const errorBox = document.querySelector('.model-error');

const MODEL_BUILD = '2026-06-19-reliable-loader';
const modelFiles = {
  web: { url: 'assets/model/camp-real-web-nodraco.glb', label: '高清网页模型', size:
'61.5MB' },
  lite: { url: 'assets/model/camp-real-web.glb', label: '极速轻量模型', size: '5MB' },
  full: { url: 'assets/model/camp-real.glb', label: '完整真实模型', size: '547MB' }
};

if (!canvas) throw new Error('未找到 threeViewer 画布');
canvas.tabIndex = 0;

function showMessage(message, isError = false) {
  if (isError) {
    errorBox.textContent = message;
    errorBox.classList.add('is-visible');
    console.error(message);
  } else if (progress) {
    progress.textContent = message;
  }
}

function hideProgress() {
  progressBoxes.forEach((node) => {
    node.classList.add('is-hidden');
    node.setAttribute('aria-hidden', 'true');
  });
}

function showProgress() {
  progressBoxes.forEach((node) => {
    node.classList.remove('is-hidden');
    node.removeAttribute('aria-hidden');
  });
}

function clearError() {
  errorBox.textContent = '';
  errorBox.classList.remove('is-visible');
}
```

```
window.addEventListener('error', (event) => showMessage(`页面脚本错误：${event.message}`,
true));
window.addEventListener('unhandledrejection', (event) => showMessage(`模型加载异常：
${event.reason?.message || event.reason}`, true));

const renderer = new THREE.WebGLRenderer({ canvas, antialias: true, powerPreference:
'high-performance' });
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 1.5));
renderer.setClearColor(0x10231d, 1);
renderer.outputColorSpace = THREE.SRGBColorSpace;
renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.toneMappingExposure = 1.18;

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x10231d);
scene.fog = new THREE.Fog(0x10231d, 110, 760);

const camera = new THREE.PerspectiveCamera(42, 1, 0.1, 100000);
camera.position.set(16, 14, 18);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
controls.dampingFactor = 0.075;
controls.autoRotate = true;
controls.autoRotateSpeed = 0.28;
controls.target.set(0, 0, 0);
controls.enablePan = true;
controls.screenSpacePanning = true;
controls.panSpeed = 1.7;
controls.zoomToCursor = true;
controls.keyPanSpeed = 42;

scene.add(new THREE.HemisphereLight(0xffffff, 0x24452c, 2.1));
const keyLight = new THREE.DirectionalLight(0xfff0cf, 3.1);
keyLight.position.set(100, 180, 120);
scene.add(keyLight);
const rimLight = new THREE.DirectionalLight(0x80e8ff, 1.15);
rimLight.position.set(-120, 90, -80);
scene.add(rimLight);

const floor = new THREE.Mesh(
  new THREE.CircleGeometry(1000, 96),
  new THREE.MeshStandardMaterial({ color: 0x315b35, roughness: 0.9 })
);
floor.rotation.x = -Math.PI / 2;
floor.visible = false;
scene.add(floor);

const loader = new GLTFLoader();
const dracoLoader = new DRACOLoader();
dracoLoader.setDecoderPath('./vendor/three/');
dracoLoader.setDecoderConfig({ type: 'wasm' });
```

```
loader.setDRACOLoader(dracoLoader);
let activeModel = null;
let activeSphere = null;
let activeKind = 'web';
let helperGroup = null;
let modelBox = null;
let coreBox = null;
let loadRequestId = 0;
const focusState = { core: true, ratio: 0.38 };
const pressedKeys = new Set();
let renderMode = 'enhanced';

function setActiveButton(selector, activeValue) {
  document.querySelectorAll(selector).forEach((button) => {
    const datasetKey = selector.match(/data-([a-z]+)/)?.[1];
    button.classList.toggle('is-active', datasetKey ? button.dataset[datasetKey] ===
activeValue : false);
  });
}

function refreshModeButtons() {
  setActiveButton('[data-crop]', focusState.core ? 'core' : 'all');
  setActiveButton('[data-load]', activeKind);
  setActiveButton('[data-render]', renderMode);
}

function resize() {
  const rect = canvas.getBoundingClientRect();
  const width = Math.max(1, rect.width);
  const height = Math.max(1, rect.height);
  if (canvas.width !== Math.floor(width * renderer.getPixelRatio()) || canvas.height !==
Math.floor(height * renderer.getPixelRatio())) {
    renderer.setSize(width, height, false);
    camera.aspect = width / height;
    camera.updateProjectionMatrix();
  }
}

function beautifyModel(root) {
  root.traverse((node) => {
    if (!node.isMesh) return;
    node.frustumCulled = false;
    node.renderOrder = 1;
    const materials = Array.isArray(node.material) ? node.material : [node.material];
    materials.forEach((material) => {
      if (!material) return;
      material.side = THREE.DoubleSide;
      material.transparent = false;
      material.opacity = 1;
      material.depthWrite = true;
      material.depthTest = true;
      material.roughness = Math.max(material.roughness ?? 0.8, 0.72);
```

```
      material.needsUpdate = true;
    });
  });
}

function applyRenderMode(mode = renderMode) {
  renderMode = mode;
  if (!activeModel) return;
  activeModel.traverse((node) => {
    if (!node.isMesh) return;
    if (!node.userData.originalMaterial) node.userData.originalMaterial = node.material;
    if (mode === 'material') {
      node.material = node.userData.originalMaterial;
    } else if (mode === 'clay') {
      node.material = new THREE.MeshStandardMaterial({ color: 0xe7dfc7, roughness: 0.86,
metalness: 0.02, side: THREE.DoubleSide });
    } else {
      const source = Array.isArray(node.userData.originalMaterial) ?
node.userData.originalMaterial[0] : node.userData.originalMaterial;
      const color = source?.color?.clone?.() || new THREE.Color(0xf1f5dc);
      node.material = new THREE.MeshStandardMaterial({
        color: color.lerp(new THREE.Color(0xbff0cc), 0.16),
        roughness: 0.68,
        metalness: 0.02,
        side: THREE.DoubleSide,
        emissive: new THREE.Color(0x10231d),
        emissiveIntensity: 0.05
      });
    }
  });
  refreshModeButtons();
}

function expandFlatBox(box, padding = 2.5) {
  return box.clone().expandByVector(new THREE.Vector3(padding, padding * 0.25, padding));
}

function computeCoreBox(box, root = activeModel) {
  if (!root) {
    const center = box.getCenter(new THREE.Vector3());
    const size = box.getSize(new THREE.Vector3());
    return new THREE.Box3().setFromCenterAndSize(center, new
THREE.Vector3(Math.max(size.x * focusState.ratio, 18), Math.max(size.y * 1.2, 8),
Math.max(size.z * focusState.ratio, 18)));
  }

  const modelSize = box.getSize(new THREE.Vector3());
  const candidates = [];
  root.updateMatrixWorld(true);
  root.traverse((node) => {
    if (!node.isMesh || !node.geometry) return;
    const meshBox = new THREE.Box3().setFromObject(node);
    if (meshBox.isEmpty()) return;
    const size = meshBox.getSize(new THREE.Vector3());
```

```
    const footprint = size.x * size.z;
    const isWholeTerrain = size.x > modelSize.x * 0.32 || size.z > modelSize.z * 0.32;
    const isTinyNoise = footprint < 0.01;
    if (!isWholeTerrain && !isTinyNoise) candidates.push(meshBox);
  });

  if (!candidates.length) {
    const center = box.getCenter(new THREE.Vector3());
    const size = box.getSize(new THREE.Vector3());
    return new THREE.Box3().setFromCenterAndSize(center, new
THREE.Vector3(Math.max(size.x * focusState.ratio, 18), Math.max(size.y * 1.2, 8),
Math.max(size.z * focusState.ratio, 18)));
  }

  const seed = candidates.reduce((best, current) => {
    const bestSize = best.getSize(new THREE.Vector3());
    const currentSize = current.getSize(new THREE.Vector3());
    return currentSize.x * currentSize.z > bestSize.x * bestSize.z ? current : best;
  }, candidates[0]);
  const seedCenter = seed.getCenter(new THREE.Vector3());
  const clusterRadius = Math.max(Math.min(modelSize.x, modelSize.z) * 0.16, 7);
  const cluster = new THREE.Box3().makeEmpty();

  candidates.forEach((candidate) => {
    const center = candidate.getCenter(new THREE.Vector3());
    const distance = Math.hypot(center.x - seedCenter.x, center.z - seedCenter.z);
    if (distance <= clusterRadius || seed.intersectsBox(expandFlatBox(candidate, 3)))
cluster.union(candidate);
  });

  return expandFlatBox(cluster, Math.max(clusterRadius * 0.12, 1.4));
}

function updateFocusArea(root) {
  if (!modelBox) return;
  root.traverse((node) => {
    if (node.isMesh) node.visible = true;
  });
  coreBox = computeCoreBox(modelBox, root);
  activeSphere = (focusState.core ? coreBox : modelBox).getBoundingSphere(new
THREE.Sphere());
}

function addVisibilityHelpers(root) {
  if (helperGroup) scene.remove(helperGroup);
  helperGroup = new THREE.Group();
  helperGroup.name = 'camp_visual_helpers';

  const box = getVisibleMeshBox(root);
  if (!box.isEmpty()) {
    const helper = new THREE.Box3Helper(box, 0x70e1b4);
    helper.material.transparent = true;
    helper.material.opacity = 0.35;
    helperGroup.add(helper);
```

```
  }

  scene.add(helperGroup);
}

function getVisibleMeshBox(root) {
  const box = new THREE.Box3().makeEmpty();
  root.updateMatrixWorld(true);
  root.traverse((node) => {
    if (!node.isMesh || !node.geometry || !node.visible) return;
    box.union(new THREE.Box3().setFromObject(node));
  });
  return box;
}

function centerAndFrame(root, view = 'overview') {
  const box = new THREE.Box3().setFromObject(root);
  if (box.isEmpty()) throw new Error('模型为空，无法展示');

  const center = box.getCenter(new THREE.Vector3());
  const size = box.getSize(new THREE.Vector3());
  const maxSize = Math.max(size.x, size.y, size.z);
  if (maxSize > 0) {
    const scale = 80 / maxSize;
    root.scale.setScalar(scale);
    root.position.copy(center).multiplyScalar(-scale);
  }
  root.updateMatrixWorld(true);

  modelBox = new THREE.Box3().setFromObject(root).clone();
  updateFocusArea(root);

  floor.visible = true;
  floor.position.y = (modelBox || coreBox).min.y - 0.05;
  floor.scale.setScalar(Math.max(120, activeSphere.radius * 4) / 900);

  addVisibilityHelpers(root);
  frameView(view);
}

function frameView(view = 'overview') {
  if (!activeSphere) return;
  const radius = Math.max(activeSphere.radius, 1);
  const target = activeSphere.center;
  const distance = Math.max(radius * 2.15, 26);
  const positions = {
    overview: [distance * 0.78, distance * 0.72, distance * 0.92],
    bird: [distance * 0.34, distance * 1.18, distance * 0.48],
    front: [0, distance * 0.34, distance * 1.05],
    top: [0, distance * 1.65, distance * 0.001],
```

```
    low: [distance * 0.92, radius * 0.28, distance * 0.56],
    reset: [distance * 0.78, distance * 0.72, distance * 0.92]
  };
  const [x, y, z] = positions[view] || positions.overview;
  camera.position.set(target.x + x, target.y + y, target.z + z);
  camera.near = Math.max(radius / 1000, 0.05);
  camera.far = Math.max(radius * 28, 10000);
  camera.updateProjectionMatrix();
  controls.target.copy(target);
  controls.minDistance = radius * 0.12;
  controls.maxDistance = radius * 10;
  controls.update();
}

function countModel(root) {
  let meshes = 0;
  let triangles = 0;
  root.traverse((node) => {
    if (!node.isMesh || !node.geometry) return;
    meshes += 1;
    triangles += node.geometry.index ? node.geometry.index.count / 3 :
node.geometry.attributes.position.count / 3;
  });
  return { meshes, triangles: Math.round(triangles) };
}

async function loadModel(kind = 'web') {
  const requestId = ++loadRequestId;
  const fallbackKind = kind === 'full' ? 'web' : 'lite';
  const file = modelFiles[kind] || modelFiles.web;
  activeKind = kind;
  clearError();
  showProgress();
  showMessage(`正在加载${file.label}（${file.size}）...`);

  try {
    await loadModelFile(kind, file, requestId);
  } catch (error) {
    if (requestId !== loadRequestId) return;
    if (kind !== 'lite' && modelFiles[fallbackKind]) {
      console.warn(`${file.label}加载失败，自动切换到${modelFiles[fallbackKind].label}`,
error);
      showMessage(`${file.label}加载较慢或失败，正在自动切换到
${modelFiles[fallbackKind].label}...`);
      try {
        await loadModel(fallbackKind);
        return;
      } catch (fallbackError) {
        showMessage(`备用模型加载失败：${fallbackError.message || fallbackError}`, true);
      }
    } else {
      showMessage(`模型加载失败：${error.message || error}. 请确认通过"启动网站.bat"打开，不要
直接双击 HTML 文件。`, true);
    }
```

```
    showMessage('模型加载失败，请点击"极速轻量版"重试。');
  }
}

async function loadModelFile(kind, file, requestId) {
  if (window.location.protocol === 'file:') {
    throw new Error('当前是 file:// 打开，浏览器会拦截 3D 模型。请使用"启动网站.bat"或
http://127.0.0.1:8787/model-test.html');
  }

  const url = `${file.url}?v=${MODEL_BUILD}`;
  await checkModelAsset(url, file);

  return new Promise((resolve, reject) => {
    const timeout = window.setTimeout(() => {
      reject(new Error(`${file.label}超过 25 秒仍未完成加载`));
    }, kind === 'full' ? 90000 : 25000);

    loader.load(
      url,
      (gltf) => {
        window.clearTimeout(timeout);
        if (requestId !== loadRequestId) {
          resolve(gltf);
          return;
        }
        if (activeModel) scene.remove(activeModel);
        activeModel = gltf.scene;
        activeKind = kind;
        beautifyModel(activeModel);
        scene.add(activeModel);
        centerAndFrame(activeModel);
        applyRenderMode(renderMode);
        const { meshes, triangles } = countModel(activeModel);
        stats.textContent = `${file.label} · ${meshes} 个网格 · 约
${triangles.toLocaleString()} 个三角面 · 鼠标旋转/滚轮缩放/右键平移，WASD/QE 可微调视角`;
        hideProgress();
        refreshModeButtons();
        resolve(gltf);
      },
      (xhr) => {
        if (xhr.total) {
          showMessage(`${file.label}加载中 ${Math.round((xhr.loaded / xhr.total) *
100)}% · ${(xhr.loaded / 1024 / 1024).toFixed(1)}MB / ${(xhr.total / 1024 /
1024).toFixed(1)}MB`);
        } else {
          showMessage(`${file.label}加载中 ${(xhr.loaded / 1024 / 1024).toFixed(1)}MB`);
        }
      },
      (error) => {
        window.clearTimeout(timeout);
        reject(error);
      }
    );
```

```
  });
}

async function checkModelAsset(url, file) {
  const response = await fetch(url, { method: 'HEAD', cache: 'no-store' });
  if (!response.ok) throw new Error(`${file.label}文件访问失败：HTTP ${response.status}`);
  const bytes = Number(response.headers.get('content-length') || 0);
  if (bytes > 0) showMessage(`已找到${file.label} ${(bytes / 1024 / 1024).toFixed(1)}MB，
开始解析...`);
}

function moveCameraByKeys() {
  if (!activeSphere || pressedKeys.size === 0) return;
  const speed = Math.max(activeSphere.radius * 0.016, 0.08);
  const forward = new THREE.Vector3();
  const right = new THREE.Vector3();
  camera.getWorldDirection(forward);
  forward.y = 0;
  forward.normalize();
  right.crossVectors(forward, camera.up).normalize();

  const movement = new THREE.Vector3();
  if (pressedKeys.has('KeyW') || pressedKeys.has('ArrowUp')) movement.add(forward);
  if (pressedKeys.has('KeyS') || pressedKeys.has('ArrowDown')) movement.sub(forward);
  if (pressedKeys.has('KeyA') || pressedKeys.has('ArrowLeft')) movement.sub(right);
  if (pressedKeys.has('KeyD') || pressedKeys.has('ArrowRight')) movement.add(right);
  if (pressedKeys.has('KeyQ')) movement.y -= 1;
  if (pressedKeys.has('KeyE')) movement.y += 1;
  if (movement.lengthSq() === 0) return;
  movement.normalize().multiplyScalar(speed);
  camera.position.add(movement);
  controls.target.add(movement);
}

function animate() {
  resize();
  moveCameraByKeys();
  controls.update();
  renderer.render(scene, camera);
  requestAnimationFrame(animate);
}

function normalizeCameraKey(event) {
  const key = `${event.code || event.key || ''}`.toLowerCase();
  const map = {
    keyw: 'KeyW', w: 'KeyW', arrowup: 'ArrowUp',
    keys: 'KeyS', s: 'KeyS', arrowdown: 'ArrowDown',
    keya: 'KeyA', a: 'KeyA', arrowleft: 'ArrowLeft',
    keyd: 'KeyD', d: 'KeyD', arrowright: 'ArrowRight',
    keyq: 'KeyQ', q: 'KeyQ', keye: 'KeyE', e: 'KeyE'
  };
```

```
  return map[key] || event.code;
}

document.querySelectorAll('[data-view]').forEach((button) => {
  button.addEventListener('click', () => {
    controls.autoRotate = false;
    frameView(button.dataset.view);
  });
});

document.querySelectorAll('[data-load]').forEach((button) => {
  button.addEventListener('click', () => loadModel(button.dataset.load));
});

document.querySelectorAll('[data-render]').forEach((button) => {
  button.addEventListener('click', () => applyRenderMode(button.dataset.render));
});

document.querySelectorAll('[data-crop]').forEach((button) => {
  button.addEventListener('click', () => {
    focusState.core = button.dataset.crop === 'core';
    if (activeModel) {
      updateFocusArea(activeModel);
      addVisibilityHelpers(activeModel);
      frameView('overview');
    }
    refreshModeButtons();
  });
});

canvas.addEventListener('click', () => canvas.focus());
window.addEventListener('keydown', (event) => {
  const code = normalizeCameraKey(event);
  const movementCodes = ['KeyW', 'KeyA', 'KeyS', 'KeyD', 'KeyQ', 'KeyE', 'ArrowUp',
'ArrowDown', 'ArrowLeft', 'ArrowRight'];
  if (!movementCodes.includes(code)) return;
  pressedKeys.add(code);
  controls.autoRotate = false;
  event.preventDefault();
  event.stopPropagation();
}, { capture: true });

window.addEventListener('keyup', (event) => {
  pressedKeys.delete(normalizeCameraKey(event));
}, { capture: true });
window.addEventListener('blur', () => pressedKeys.clear());

refreshModeButtons();
window.__camp3d = {
  camera,
  controls,
```

```
  get coreBox() { return coreBox; },
  get modelBox() { return modelBox; },
  get activeSphere() { return activeSphere; },
  countVisibleMeshes() {
    let total = 0;
    let visible = 0;
    activeModel?.traverse((node) => {
      if (node.isMesh) {
        total += 1;
        if (node.visible) visible += 1;
      }
    });
    return { total, visible };
  },
  getBounds() {
    const coreSize = coreBox?.getSize(new THREE.Vector3()).toArray() || null;
    const coreCenter = coreBox?.getCenter(new THREE.Vector3()).toArray() || null;
    const modelSize = modelBox?.getSize(new THREE.Vector3()).toArray() || null;
    return { coreSize, coreCenter, modelSize };
  },
  frameView
};

loadModel('web');
animate();

  (() => {
  const root = document.documentElement;
  const reduce = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
  root.classList.add('is-booting');
  const rotations = [-4, 2, -1, 5, -3, 1];

  function makeLoader() {
    if (reduce) return null;
    const loader = document.createElement('div');
    loader.className = 'campos-loader';
    loader.innerHTML = '<div class="camp-loader-map" aria-hidden="true"><svg viewBox="0
0 900 520"><path class="camp-loader-terrain t1" d="M-20 360 C120 280 205 380 330 310
C470 230 570 280 700 198 C790 145 850 168 930 112 L930 560 L-20 560Z"/><path
class="camp-loader-terrain t2" d="M-20 420 C130 350 245 455 390 368 C525 288 650 365 770
282 C840 234 885 248 930 214 L930 560 L-20 560Z"/><path class="camp-loader-route"
d="M105 300 C210 238 298 306 405 248 C530 180 618 225 735 160"/><g class="camp-loader-
nodes"><circle cx="192" cy="263" r="9"/><circle cx="406" cy="248" r="9"/><circle
cx="612" cy="220" r="9"/><circle cx="735" cy="160" r="9"/></g></svg></div><div
class="campos-loader__stack"><div class="campos-
loader__code"><span>\u8425\u5730\u6570\u5b57\u5b6a\u751f\u7cfb\u7edf</span><span>\u6b63\
u5728\u751f\u6210\u8fd0\u8425\u5730\u5f62</span></div><h1 class="campos-
loader__title">\u6797\u95f4\u5f8b\u52a8</h1><p class="campos-loader__sub">Digital camp
operating system</p><div class="campos-loader__bar"><i></i></div></div><div
class="campos-loader__tiles"></div>';
    const tiles = loader.querySelector('.campos-loader__tiles');
    for (let i = 0; i < 12; i += 1) {
      const tile = document.createElement('span');
      tile.style.setProperty('--i', i);
      tile.style.setProperty('--r', rotations[i % rotations.length] + 'deg');
```

```
    tiles.appendChild(tile);
  }
  document.body.prepend(loader);
  return loader;
}

function addWipe() {
  if (reduce) return null;
```

```
    const wipe = document.createElement('div');
    wipe.className = 'campos-page-wipe';
    for (let i = 0; i < 6; i += 1) {
      const panel = document.createElement('span');
      panel.style.setProperty('--i', i);
      wipe.appendChild(panel);
    }
    document.body.appendChild(wipe);
    return wipe;
  }

  function splitText(target) {
    if (!target || target.dataset.camposSplit === 'done') return;
    const text = target.textContent.trim();
    if (!text || text.length > 90) return;
    target.dataset.camposSplit = 'done';
    target.setAttribute('aria-label', text);
    target.textContent = '';
    Array.from(text).forEach((letter, index) => {
      const span = document.createElement('span');
      span.className = 'char';
      span.style.setProperty('--char-index', index);
      span.textContent = letter === ' ' ? '\u00A0' : letter;
      target.appendChild(span);
    });
  }

  function prepareMotion(rootNode = document) {
    rootNode.querySelectorAll('.campos-loader__title').forEach(splitText);
    const selector = '.hero > div, .dashboard-card, .section, .card, .value-
card, .panel, .form, .download-card, .feature-card, .stats article, .timeline
article, .login-card, .row, .metric, .ops-item, .three-viewer-shell';
    rootNode.querySelectorAll(selector).forEach((element, index) => {
      if (element.classList.contains('campos-reveal')) return;
      element.classList.add('campos-reveal');
      element.style.transitionDelay = Math.min(index % 7, 6) * 45 + 'ms';
      const modes = ['clip', 'left', 'right', 'slam', 'leaf', 'water'];
      element.dataset.motion = modes[index % modes.length];
    });
    rootNode.querySelectorAll('a, button').forEach(el => el.classList.add('campos-
magnetic'));
    rootNode.querySelectorAll('.card,.value-card,.download-card,.dashboard-
card,.panel,.metric').forEach(el => el.classList.add('campos-tilt'));
  }

  function observeReveals() {
    const observer = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (!entry.isIntersecting) return;
        entry.target.classList.add('is-in');
        observer.unobserve(entry.target);
      });
    }, { threshold: 0.16, rootMargin: '0px 0px -9% 0px' });
    document.querySelectorAll('.campos-reveal, [data-campos-split]').forEach(el =>
observer.observe(el));
```

```
    return observer;
  }

  function addProgress() {
    const progress = document.createElement('div');
    progress.className = 'campos-progress';
    progress.innerHTML = '<i></i>';
    document.body.appendChild(progress);
    const bar = progress.querySelector('i');
    const update = () => {
      const max = Math.max(1, document.documentElement.scrollHeight - innerHeight);
      bar.style.transform = 'scaleX(' + Math.min(scrollY / max, 1) + ')';
    };
    addEventListener('scroll', update, { passive: true });
    addEventListener('resize', update);
    update();
  }

  function addCursor() {
    if (reduce || matchMedia('(pointer: coarse)').matches) return;
    const cursor = document.createElement('div');
    cursor.className = 'campos-kinetic';
    document.body.appendChild(cursor);
    let x = innerWidth / 2, y = innerHeight / 2, tx = x, ty = y;
    addEventListener('pointermove', event => { tx = event.clientX; ty = event.clientY;
cursor.style.opacity = '1'; }, { passive: true });
    document.addEventListener('pointerover', event => { if
(event.target.closest('a,button,.campos-tilt')) cursor.classList.add('is-hot'); });
    document.addEventListener('pointerout', event => { if
(event.target.closest('a,button,.campos-tilt')) cursor.classList.remove('is-hot'); });
    const loop = () => { x += (tx - x) * .18; y += (ty - y) * .18;
cursor.style.transform = 'translate(' + (x - 9) + 'px,' + (y - 9) + 'px) rotate(45deg)';
requestAnimationFrame(loop); };
    loop();
  }

  function bindMagnetic() {
    if (reduce || matchMedia('(pointer: coarse)').matches) return;
    document.addEventListener('pointermove', event => {
      const target = event.target.closest('.campos-magnetic');
      if (!target) return;
      const rect = target.getBoundingClientRect();
      const dx = event.clientX - (rect.left + rect.width / 2);
      const dy = event.clientY - (rect.top + rect.height / 2);
      target.style.transform = 'translate(' + (dx * .12) + 'px,' + (dy * .16) + 'px)';
    }, { passive: true });
    document.addEventListener('pointerout', event => {
      const target = event.target.closest('.campos-magnetic');
      if (target) target.style.transform = '';
    });
  }

  function bindTilt() {
    if (reduce || matchMedia('(pointer: coarse)').matches) return;
    document.addEventListener('pointermove', event => {
```

```
      const card = event.target.closest('.campos-tilt');
      if (!card) return;
      const rect = card.getBoundingClientRect();
      const px = (event.clientX - rect.left) / rect.width - .5;
      const py = (event.clientY - rect.top) / rect.height - .5;
      card.style.transform = 'perspective(900px) rotateX(' + (py * -4) + 'deg) rotateY('
+ (px * 5) + 'deg) translateY(-3px)';
    }, { passive: true });
    document.addEventListener('pointerout', event => {
      const card = event.target.closest('.campos-tilt');
      if (card) card.style.transform = '';
    });
  }

  function addOrbit() {
    const hero = document.querySelector('.hero, main, body');
    if (hero && !document.querySelector('.campos-orbit')) {
      const orbit = document.createElement('div');
      orbit.className = 'campos-orbit';
      hero.appendChild(orbit);
    }
  }

  function animateLinks(wipe) {
    if (!wipe) return;
    document.addEventListener('click', event => {
      const link = event.target.closest('a[href]');
      if (!link || link.target || link.hasAttribute('download')) return;
      const href = link.getAttribute('href');
      if (!href || href.startsWith('#') || href.startsWith('mailto:') ||
href.startsWith('tel:')) return;
      event.preventDefault();
      wipe.classList.remove('is-active');
      void wipe.offsetWidth;
      wipe.classList.add('is-active');
      setTimeout(() => { location.href = link.href; }, 360);
    });
  }



  function addForestMotion() {
    let lastRipple = 0;
    document.addEventListener('pointerdown', event => {
      if (Date.now() - lastRipple < 180) return;
      lastRipple = Date.now();
      const ripple = document.createElement('span');
      ripple.className = 'forest-ripple';
      ripple.style.left = event.clientX + 'px';
      ripple.style.top = event.clientY + 'px';
      document.body.appendChild(ripple);
      setTimeout(() => ripple.remove(), 1300);
```

```
    });
    if (!matchMedia('(prefers-reduced-motion: reduce)').matches) {
      setInterval(() => {
        if (document.hidden || Math.random() > .72) return;
        const leaf = document.createElement('span');
        leaf.className = 'forest-leaf';
        leaf.style.setProperty('--x', Math.round(Math.random() * innerWidth) + 'px');
        leaf.style.setProperty('--drift', Math.round((Math.random() - .5) * 220) + 'px');
        leaf.style.left = '0';
        leaf.style.top = '0';
        document.body.appendChild(leaf);
        setTimeout(() => leaf.remove(), 6200);
      }, 1300);
    }
    document.querySelectorAll('video').forEach(video => {
      video.addEventListener('play', () => { video.classList.add('is-playing');
video.closest('.video-shell')?.classList.add('is-cinematic'); });
      video.addEventListener('pause', () => video.closest('.video-
shell')?.classList.remove('is-cinematic'));
      video.addEventListener('ended', () => video.closest('.video-
shell')?.classList.remove('is-cinematic'));
    });
    document.querySelectorAll('[data-video-play]').forEach(button => {
      button.addEventListener('click', () => {
        const video = document.querySelector('.tour-video, video');
        if (!video) return;
        video.scrollIntoView({ behavior: 'smooth', block: 'center' });
        video.play().catch(() => { video.controls = true; });
      });
    });
    const parallaxItems = document.querySelectorAll('.panel-image,.video-shell,.three-
viewer-shell,.hero-card');
    const update = () => {
      parallaxItems.forEach((item, index) => {
        const rect = item.getBoundingClientRect();
        const center = rect.top + rect.height / 2;
        const offset = (center - innerHeight / 2) / innerHeight;
        item.style.setProperty('--forest-parallax', offset.toFixed(3));
        if (!item.matches(':hover')) item.style.transform = 'translateY(' + (offset * -
18 * (index % 2 ? .65 : 1)) + 'px) rotate(' + (offset * (index % 2 ? -1 : 1)) + 'deg)';
      });
    };
    addEventListener('scroll', update, { passive: true });
    addEventListener('resize', update);
    update();
  }



  function hardenDynamicVisibility() {
    const markReady = () => {
      if (location.pathname.includes('/admin/'))
document.documentElement.classList.add('admin-ready');
      if (location.pathname.includes('/app/'))
document.documentElement.classList.add('app-ready');
```

```
      document.querySelectorAll('#workspace .campos-reveal, main .metric, main .card,
main .value-card, main .ops-item, .table .row').forEach((element, index) => {
        if (!element.classList.contains('campos-reveal')) element.classList.add('campos-
reveal');
```

```
        window.setTimeout(() => element.classList.add('is-in'), Math.min(index, 18) *
24);
      });
    };
    markReady();
    window.setTimeout(markReady, 500);
    window.setTimeout(markReady, 1400);
    window.setTimeout(markReady, 2600);
  }

  function boot() {
    const loader = makeLoader();
    const wipe = addWipe();
    prepareMotion();
    const revealObserver = observeReveals();
    addProgress();
    addCursor();
    bindMagnetic();
    bindTilt();
    addOrbit();
    // forest motion disabled in final black clarity mode
    animateLinks(wipe);
    hardenDynamicVisibility();

    const mutationObserver = new MutationObserver((mutations) => {
      mutations.forEach(mutation => {
        mutation.addedNodes.forEach(node => {
          if (!(node instanceof Element)) return;
          prepareMotion(node);
          node.querySelectorAll && node.querySelectorAll('.campos-reveal, [data-campos-
split]').forEach(el => { revealObserver.observe(el); setTimeout(() =>
el.classList.add('is-in'), 80); });
          if (node.matches && node.matches('.campos-reveal, [data-campos-split]'))
{ revealObserver.observe(node); setTimeout(() => node.classList.add('is-in'), 80); }
        });
      });
    });
    mutationObserver.observe(document.body, { childList: true, subtree: true });

    setTimeout(() => {
      root.classList.remove('is-booting');
      root.classList.add('is-ready');
      document.querySelectorAll('[data-campos-split]').forEach(el =>
el.classList.add('is-in'));
      document.querySelectorAll('.hero .campos-reveal, .login-card.campos-
reveal, .panel.campos-reveal').forEach(el => el.classList.add('is-in'));
      if (loader) {
        loader.classList.add('is-exiting');
        setTimeout(() => loader.remove(), 850);
      }
    }, reduce ? 0 : 1550);
  }

  if (document.readyState === 'loading') document.addEventListener('DOMContentLoaded',
boot, { once: true });
```

```
  else boot();
})();
```

```
/* CampOS motion system ? shared local layer */
:root{--motion-black:#050504;--motion-paper:#e1d6ce;--motion-red:#b9282c;--motion-
lime:#d8ef69;--motion-line:rgba(225,214,206,.18)}
html.is-booting{overflow:hidden}html.is-ready body{--boot-ready:1}.campos-
loader{position:fixed;inset:0;z-index:9999;display:grid;place-
items:center;background:#050504;color:var(--motion-paper);overflow:hidden;pointer-
events:none}.campos-loader:before{content:"";position:absolute;inset:-
20%;background:linear-gradient(90deg,rgba(225,214,206,.055) 1px,transparent 1px),linear-
gradient(rgba(225,214,206,.045) 1px,transparent 1px);background-size:64px
64px;transform:rotate(-7deg) scale(1.15);animation:camposGrid 1.5s cubic-
bezier(.2,.8,.2,1) both}.campos-
loader:after{content:"";position:absolute;inset:0;background:linear-
gradient(90deg,transparent,rgba(216,239,105,.18),transparent);transform:translateX(-120%)
skewX(-18deg);animation:camposScan 1.35s .18s cubic-bezier(.22,1,.36,1) both}.campos-
loader__stack{position:relative;z-
index:2;width:min(760px,86vw);display:grid;gap:18px}.campos-
loader__code{display:flex;align-items:center;justify-content:space-
between;gap:16px;color:rgba(216,239,105,.82);font-size:12px;font-weight:950;letter-
spacing:.22em;text-transform:uppercase}.campos-loader__title{margin:0;font-
family:Georgia,"Times New Roman",serif;font-size:clamp(58px,12vw,158px);line-
height:.76;letter-spacing:-.09em;color:var(--motion-paper);text-shadow:10px 12px 0
rgba(185,40,44,.76);clip-path:inset(0 0 100% 0);animation:camposTitleCut .92s .25s
cubic-bezier(.16,1,.3,1) forwards}.campos-loader__bar{height:16px;border:1px solid
rgba(225,214,206,.22);background:rgba(225,214,206,.06);box-shadow:9px 10px 0
rgba(0,0,0,.55);overflow:hidden}.campos-loader__bar
i{display:block;width:100%;height:100%;background:linear-gradient(90deg,var(--motion-
red),var(--motion-lime));transform-
origin:left;transform:scaleX(0);animation:camposLoadBar 1.35s .2s cubic-
bezier(.22,1,.36,1) forwards}.campos-loader__tiles{position:absolute;inset:auto 0 8vh
0;display:grid;grid-template-columns:repeat(12,1fr);gap:10px;padding:0
min(8vw,90px)}.campos-loader__tiles span{height:clamp(42px,8vw,110px);background:var(--
motion-red);box-shadow:8px 10px 0 rgba(0,0,0,.62);transform:translateY(120%)
rotate(var(--r,0deg));animation:camposTileUp .86s cubic-bezier(.16,1,.3,1)
forwards;animation-delay:calc(.1s + var(--i)*.035s)}.campos-loader.is-
exiting{animation:camposLoaderExit .78s cubic-bezier(.76,0,.24,1) forwards}.campos-
loader.is-exiting .campos-loader__stack{animation:camposStackOut .5s cubic-
bezier(.76,0,.24,1) forwards}.campos-page-wipe{position:fixed;inset:0;z-
index:9998;pointer-events:none;display:grid;grid-template-columns:repeat(6,1fr)}.campos-
page-wipe span{background:var(--motion-red);transform:scaleY(0);transform-
origin:bottom}.campos-page-wipe.is-active span{animation:camposWipe .72s cubic-
bezier(.76,0,.24,1) both;animation-delay:calc(var(--i)*.045s)}
[data-campos-split] .char{display:inline-block;opacity:0;transform:translateY(1.15em)
rotateX(78deg) skewY(7deg);transform-origin:0 100%;filter:blur(8px)}html.is-ready [data-
campos-split].is-in .char{animation:camposCharIn .82s cubic-bezier(.16,1,.3,1)
forwards;animation-delay:calc(var(--char-index)*22ms + var(--split-delay,0ms))}.campos-
reveal{opacity:0;transform:translate3d(0,48px,0) rotateX(7deg)
scale(.98);filter:blur(10px);transition:opacity .78s cubic-
bezier(.16,1,.3,1),transform .78s cubic-bezier(.16,1,.3,1),filter .78s ease,clip-
path .92s cubic-bezier(.16,1,.3,1)}.campos-reveal.is-
in{opacity:1;transform:none;filter:blur(0)}.campos-reveal[data-
motion="left"]{transform:translate3d(-70px,18px,0) rotateZ(-2deg)}.campos-reveal[data-
motion="right"]{transform:translate3d(70px,18px,0) rotateZ(2deg)}.campos-reveal[data-
motion="clip"]{clip-path:inset(0 100% 0 0);transform:none;filter:none}.campos-
```

```
reveal[data-motion="clip"].is-in{clip-path:inset(0 0 0 0)}.campos-reveal[data-
motion="slam"]{transform:translate3d(0,80px,0) rotate(-3deg) scale(.92)}.campos-
progress{position:fixed;left:0;top:0;z-
index:9997;width:100%;height:4px;background:rgba(225,214,206,.08);pointer-
events:none}.campos-progress i{display:block;width:100%;height:100%;background:linear-
gradient(90deg,var(--motion-red),var(--motion-lime));transform:scaleX(0);transform-
origin:left}.campos-magnetic{will-change:transform}.campos-tilt{transform-
style:preserve-3d;will-change:transform}.campos-kinetic{position:fixed;left:0;top:0;z-
index:9996;width:18px;height:18px;border:1px solid
rgba(216,239,105,.8);background:rgba(216,239,105,.08);mix-blend-mode:difference;pointer-
events:none;opacity:0;transition:opacity .2s ease,width .18s ease,height .18s
ease}.campos-kinetic.is-hot{width:42px;height:42px;border-
color:rgba(185,40,44,.95);background:rgba(185,40,44,.18)}.campos-
orbit{position:absolute;pointer-events:none;border:1px solid
rgba(216,239,105,.2);width:220px;height:220px;right:8vw;top:26vh;transform:rotate(24deg);
animation:camposOrbit 10s linear infinite}.campos-orbit:before,.campos-
orbit:after{content:"";position:absolute;width:16px;height:16px;background:var(--motion-
lime);box-shadow:5px 6px 0 rgba(185,40,44,.8)}.campos-orbit:before{left:-
8px;top:38%}.campos-orbit:after{right:14%;bottom:-8px;background:var(--motion-red)}
@media (prefers-reduced-motion:reduce){.campos-loader,.campos-page-wipe,.campos-
kinetic,.campos-orbit{display:none!important}.campos-reveal,[data-campos-
split] .char{opacity:1!important;transform:none!important;filter:none!important;animatio
n:none!important;transition:none!important}}
@media (max-width:760px){.campos-loader__tiles{grid-template-
columns:repeat(6,1fr)}.campos-kinetic{display:none}.campos-orbit{opacity:.35;right:-
80px}}
@keyframes camposGrid{from{opacity:0;transform:rotate(-7deg)
scale(1.35)}to{opacity:1;transform:rotate(-7deg) scale(1.15)}}@keyframes
camposScan{to{transform:translateX(120%) skewX(-18deg)}}@keyframes
camposTitleCut{to{clip-path:inset(0 0 0 0)}}@keyframes
camposLoadBar{to{transform:scaleX(1)}}@keyframes camposTileUp{to{transform:translateY(0)
rotate(var(--r,0deg))}}@keyframes camposLoaderExit{to{transform:translateY(-
100%);visibility:hidden}}@keyframes camposStackOut{to{opacity:0;transform:translateY(-
40px) rotateX(16deg)}}@keyframes camposWipe{0%{transform:scaleY(0);transform-
origin:bottom}45%{transform:scaleY(1);transform-
origin:bottom}100%{transform:scaleY(0);transform-origin:top}}@keyframes
camposCharIn{to{opacity:1;transform:none;filter:blur(0)}}@keyframes
camposOrbit{to{rotate:360deg}}


/* Forest motion expansion */
.campos-loader{background:radial-gradient(circle at 72%
18%,rgba(104,213,231,.16),transparent 30rem),linear-
gradient(135deg,#10231d,#183d2b)!important}.campos-loader__title{text-shadow:0 18px 50px
rgba(0,0,0,.28)!important}.campos-loader__tiles span{border-radius:999px 999px 18px
18px;background:linear-gradient(180deg,#cde879,#78ddea)!important;box-shadow:0 16px 34px
rgba(16,35,29,.22)!important}.campos-page-wipe span{background:linear-
gradient(180deg,#10231d,#85a957)!important}.campos-kinetic{border-radius:999px;border-
color:rgba(104,213,231,.9)!important;background:rgba(104,213,231,.12)!important;mix-
blend-mode:multiply}.campos-kinetic.is-hot{border-
color:rgba(242,195,90,.95)!important;background:rgba(242,195,90,.18)!important}.campos-
orbit{border-radius:48% 52% 46% 54%;border-color:rgba(104,213,231,.35)!important;box-
shadow:0 0 34px rgba(104,213,231,.18);animation:camposOrbit 14s linear infinite,
forestMorph 7s ease-in-out infinite}.campos-
orbit:before{background:#68d5e7!important}.campos-
```

```
orbit:after{background:#f2c35a!important}.forest-ripple{position:absolute;z-
index:1;pointer-events:none;border:1px solid rgba(104,213,231,.28);border-
radius:50%;width:16px;height:16px;transform:translate(-50%,-50%)
scale(.4);animation:forestRipple 1.25s ease-out forwards}.forest-leaf{position:fixed;z-
index:9995;pointer-events:none;width:10px;height:18px;border-radius:70% 0 70%
0;background:linear-
gradient(135deg,rgba(133,169,87,.9),rgba(104,213,231,.65));opacity:.72;animation:forestL
eafFall 5.8s linear forwards}.tour-video.is-playing{animation:tourFocus 1.2s cubic-
bezier(.16,1,.3,1) both}.video-shell.is-cinematic{box-shadow:0 36px 110px
rgba(16,35,29,.28)!important}.route-line{stroke-dasharray:18 12;animation:floatRoute 9s
ease-in-out infinite, routeDash 16s linear infinite}.panel-image,.video-shell,.three-
viewer-shell{will-change:transform}.campos-reveal[data-
motion="leaf"]{transform:translate3d(-26px,54px,0) rotate(-7deg)
scale(.96);filter:blur(7px)}.campos-reveal[data-
motion="water"]{transform:translate3d(36px,20px,0) scale(.98);clip-path:circle(0% at 30%
60%);filter:none}.campos-reveal[data-motion="water"].is-in{clip-path:circle(140% at 30%
60%)}
@keyframes forestMorph{50%{border-radius:54% 46% 52% 48%;transform:rotate(32deg)
scale(1.04)}}@keyframes forestRipple{to{opacity:0;transform:translate(-50%,-50%)
scale(18)}}@keyframes forestLeafFall{0%{transform:translate3d(var(--x),-40px,0)
rotate(0deg)}100%{transform:translate3d(calc(var(--x) + var(--drift)),105vh,0)
rotate(540deg);opacity:0}}@keyframes tourFocus{0%{filter:saturate(1.08)
contrast(1.03);transform:scale(1)}45%{filter:saturate(1.25)
contrast(1.08);transform:scale(1.018)}100%{filter:saturate(1.12)
contrast(1.04);transform:scale(1)}}@keyframes routeDash{to{background-position:240px 0}}


/* Final motion clarity override ? sharp kinetic motion */
.campos-loader{background:radial-gradient(circle at 72%
16%,rgba(216,239,105,.09),transparent 28rem),radial-gradient(circle at 18%
76%,rgba(185,40,44,.24),transparent 30rem),#050504!important}.campos-loader__title{text-
shadow:10px 12px 0 rgba(185,40,44,.76)!important}.campos-loader__tiles span{border-
radius:0!important;background:#b9282c!important;box-shadow:8px 10px 0
rgba(0,0,0,.62)!important}.campos-page-wipe span{background:#b9282c!important}.campos-
kinetic{border-radius:0!important;border-
color:rgba(216,239,105,.85)!important;background:rgba(216,239,105,.08)!important;mix-
blend-mode:difference!important}.campos-kinetic.is-hot{border-
color:#b9282c!important;background:rgba(185,40,44,.18)!important}.campos-orbit{border-
radius:0!important;border-color:rgba(216,239,105,.22)!important;box-
shadow:none!important;animation:camposOrbit 10s linear infinite!important}.campos-
orbit:before{background:#d8ef69!important}.campos-
orbit:after{background:#b9282c!important}.forest-leaf,.forest-
ripple{display:none!important}[data-campos-
split] .char{filter:none!important;opacity:0;transform:translateY(1em) rotateZ(-
4deg)!important}.campos-
reveal{filter:none!important;opacity:0;transform:translate3d(0,44px,0)
skewY(1.2deg);transition:opacity .62s cubic-bezier(.16,1,.3,1),transform .62s cubic-
bezier(.16,1,.3,1),clip-path .72s cubic-bezier(.16,1,.3,1)!important}.campos-reveal.is-
in{filter:none!important;opacity:1;transform:none!important}.campos-reveal[data-
motion="leaf"],.campos-reveal[data-motion="water"]{filter:none!important;clip-
path:inset(0 0 0 0)!important;transform:translate3d(0,44px,0)
skewY(1.2deg)!important}.campos-reveal[data-motion="leaf"].is-in,.campos-reveal[data-
motion="water"].is-in{transform:none!important}.campos-reveal[data-
motion="left"]{transform:translate3d(-56px,0,0) skewX(-4deg)!important}.campos-
reveal[data-motion="right"]{transform:translate3d(56px,0,0)
```

```
skewX(4deg)!important}.campos-reveal[data-motion="slam"]{transform:translate3d(0,64px,0)
scale(.96)!important}.campos-reveal[data-motion="clip"]{clip-path:inset(0 100% 0
0)!important;transform:none!important}.campos-reveal[data-motion="clip"].is-in{clip-
path:inset(0 0 0 0)!important}.tour-video.is-playing{animation:blackTourPulse .9s cubic-
bezier(.16,1,.3,1) both!important}.video-shell.is-cinematic{box-shadow:0 0 0 1px
rgba(216,239,105,.18),14px 16px 0 rgba(0,0,0,.55)!important}@keyframes
blackTourPulse{45%{transform:scale(1.012);filter:contrast(1.12)
saturate(1)!important}100%{transform:scale(1);filter:contrast(1.04)
saturate(.92)!important}}


/* Hard clarity guard ? remove all backdrop blur artifacts */
* { backdrop-filter: none !important; -webkit-backdrop-filter: none !important; }
.campos-reveal, .campos-reveal *, [data-reveal], [data-reveal] * { filter:
none !important; }
.admin-ready .section.campos-reveal,
.admin-ready .row.campos-reveal,
.admin-ready .metric.campos-reveal,
.app-ready .section.campos-reveal,
.app-ready .card.campos-reveal,
.app-ready .value-card.campos-reveal,
.app-ready .metric.campos-reveal,
.app-ready .ops-item.campos-reveal {
  opacity: 1 !important;
  transform: none !important;
  filter: none !important;
}

  <!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>营地真实 SKP 模型丨3D 数字孪生</title>
  <style>
    :root{--black:#050504;--paper:#e1d6ce;--red:#b9282c;--lime:#d8ef69;--
muted:rgba(225,214,206,.68);--line:rgba(225,214,206,.18)}
    *{box-sizing:border-
box}html,body{margin:0;height:100%;overflow:hidden;background:var(--black);font-
family:Microsoft YaHei UI,Microsoft YaHei,system-ui,sans-serif;color:var(--paper)}
    body:before{content:"";position:fixed;inset:0;z-index:1;pointer-
events:none;background:radial-gradient(circle at 72%
12%,rgba(216,239,105,.12),transparent 28rem),radial-gradient(circle at 18%
78%,rgba(185,40,44,.22),transparent 30rem),linear-gradient(rgba(225,214,206,.045)
1px,transparent 1px),linear-gradient(90deg,rgba(225,214,206,.04) 1px,transparent
1px);background-size:auto,auto,72px 72px,72px 72px;mix-blend-mode:screen}

#threeViewer{width:100vw;height:100vh;display:block;background:#020202;outline:none}.pan
el{position:fixed;left:18px;top:18px;z-index:2;width:min(455px,calc(100vw -
36px));color:var(--paper);background:rgba(5,5,4,.78);padding:16px 18px;border:1px solid
var(--line);border-radius:0;backdrop-filter:blur(18px);box-shadow:10px 12px 0
rgba(0,0,0,.48)}
    .panel:before{content:"CAMPOS / REAL SKP DIGITAL TWIN";display:block;margin-
bottom:10px;color:var(--lime);font-size:11px;font-weight:950;letter-spacing:.2em}.panel
strong{display:block;font-family:Georgia,"Times New Roman",serif;font-size:28px;letter-
```

```
spacing:-.06em}.panel .hint{font-size:12px;color:var(--muted);margin-top:6px;line-
height:1.7}.buttons{position:fixed;right:18px;bottom:18px;display:flex;flex-
wrap:wrap;justify-content:flex-end;max-width:min(820px,calc(100vw - 36px));gap:8px;z-
index:2}button{border:1px solid rgba(225,214,206,.18);border-radius:0;padding:10px
13px;font-weight:900;cursor:pointer;color:var(--paper);background:rgba(5,5,4,.76);box-
shadow:6px 7px 0 rgba(0,0,0,.44);transition:transform .18s ease,box-shadow .18s
ease,background .18s ease}button:hover,button.is-active{transform:translate(-2px,-
2px);box-shadow:9px 10px 0 rgba(0,0,0,.56);background:var(--lime);color:var(--
black)}.model-progress{position:fixed;left:50%;bottom:92px;transform:translateX(-50%);z-
index:3;color:var(--black);background:var(--lime);padding:11px 16px;border:1px solid
rgba(225,214,206,.22);border-radius:0;font-weight:950;box-shadow:8px 9px 0
rgba(0,0,0,.48);max-width:min(760px,calc(100vw - 36px));text-align:center}.model-
progress.is-hidden{display:none}.model-stats{margin-top:8px;color:var(--muted);font-
size:13px}.model-
error{position:fixed;left:18px;right:18px;top:116px;display:none;color:var(--
paper);background:rgba(185,40,44,.9);padding:14px;border:1px solid
rgba(225,214,206,.2);border-radius:0;z-index:4;box-shadow:8px 9px 0
rgba(0,0,0,.48)}.model-error.is-visible{display:block}@media(max-
width:720px){.buttons{left:18px;right:18px;justify-content:flex-start}.panel
strong{font-size:22px}}
  </style>
  <link rel="stylesheet" href="assets/campos-motion.css" />
```

```
</head>
<body>
  <canvas id="threeViewer"></canvas>
  <div class="panel">
    <strong>营地真实 SKP 模型</strong>
    <div class="hint">真实 SketchUp 模型在线展示。鼠标旋转 / 滚轮缩放 / 右键平移，WASD 平面移动视角，Q/E 升降视角。若高清模型加载慢，会自动切换极速轻量版。</div>
    <div class="model-stats"></div>
  </div>
  <div class="model-error"></div>
  <div class="model-progress"><span>真实模型加载中...</span></div>
  <div class="buttons">
    <button data-view="overview">总览</button><button data-view="bird">鸟瞰
</button><button data-view="front">正视</button><button data-view="top">俯视
</button><button data-view="low">低角度</button><button data-render="material">原材质
</button><button data-render="enhanced">增强渲染</button><button data-render="clay">陶土
</button><button data-load="lite">极速轻量版</button><button data-load="web">高清网页版
</button><button data-load="full">完整模型</button><button data-crop="core">聚焦核心
</button><button data-crop="all">全景完整</button>
  </div>
  <script type="module" src="viewer.js?v=2026-06-19-reliable-loader"></script>
  <script src="assets/campos-motion.js" defer></script>
</body>
</html>
```